

■ Plant Disease Detection & Advisory System

Complete Project Architecture, File Guide & Viva Preparation Notes

Written in Simple Vocabulary for Group Study & Professor Presentation

1. Project Overview (In Simple Words)

What is this project?

This project is an AI-powered system that helps farmers and gardeners quickly identify plant diseases from leaf photos. A user simply takes a picture of a diseased leaf and uploads it to our web interface. The system diagnoses the exact disease, shows how confident it is, and immediately gives practical organic and chemical treatment advice.

Key Achievements:

- Trained on **23 plant categories** (Apple, Tomato, Potato, Corn, Bell Pepper) covering healthy leaves and various diseases.
- Achieved **84.42% validation accuracy** using MobileNetV3 deep learning architecture.
- Built an interactive **Streamlit web application** with built-in demo sample images for quick live demonstrations.

Layer	Files Involved	Function in Simple Terms
1. Data Layer	src/dataset.py	Loads images, creates an 80/20 train-test split, and applies data augmentation (rotations/flips).
2. Model & Brain	src/model.py src/train.py	Defines the Convolutional Neural Network (CNN) and trains it to recognize disease patterns.
3. User Interface	app.py src/disease_info.py	Interactive website where users upload photos, see predictions, and read remedy instructions.
4. Utilities	predict.py run_app.bat	Tools to run tests from command line and launch the web app with a double-click.

2. File-by-File Breakdown: Use and Explanation

Here is what every file in the project does, and exactly what to say when your professor asks about it.

■ 1. app.py — The Web User Interface

- **What it is:** The main web dashboard built with Python's **Streamlit** framework.
- **What it does:** It gives the user a clean website where they can drag and drop a leaf image or pick from pre-loaded sample leaves. It sends the image to our trained model, gets the disease prediction and probability score, and displays the causes, symptoms, and organic/chemical treatments.
- **What to say to your professor:**

■ ■ 'Sir, app.py is our presentation and deployment layer. It runs a lightweight Streamlit web server that accepts leaf photos, executes PyTorch inference in real time, and renders clear diagnosis and remedies.'

■ 2. src/model.py — Neural Network Architectures

- **What it is:** The blueprint of the deep learning model.
- **What it does:** It contains the code for **MobileNetV3-Small** (Transfer Learning), a **Custom 4-block CNN built from scratch**, and **ResNet18**. We chose MobileNetV3 because it is lightweight and designed to run fast on normal computers and mobile devices while still giving high accuracy.
- **What to say to your professor:**

■■ *'In model.py, we implemented MobileNetV3 using Transfer Learning. It uses depthwise separable convolutions which drastically reduces the number of parameters, making it fast and accurate.'*

■ 3. src/dataset.py — Data Pipeline & Augmentation

- **What it is:** The data loading and preprocessing module.
- **What it does:** It scans the dataset folder, automatically splits data into **80% training** and **20% validation**, and applies **Data Augmentation** (random rotation, horizontal flip, vertical flip, brightness and contrast adjustments). This teaches the model to recognize leaves even when photos are taken from odd angles or under different lighting.
- **What to say to your professor:**

■■ *'This file builds our PyTorch DataLoader pipeline. We implemented data augmentation and normalized images with ImageNet statistics to ensure the model generalizes well and does not overfit.'*

■ 4. src/train.py — Training & Optimization Pipeline

- **What it is:** The script that actually trains the AI model.
- **What it does:** It passes images through the network, calculates error using **Cross-Entropy Loss**, and updates weights using the **AdamW optimizer**. It includes a Learning Rate Scheduler that slows down learning when improvements stall, saves the best model checkpoint to models/plant_disease_model.pth, and plots the loss/accuracy curves.
- **What to say to your professor:**

■■ *'train.py manages the training loop and backpropagation. It tracks validation loss epoch by epoch and automatically saves only the best performing checkpoint.'*

■ 5. src/disease_info.py — Agricultural Knowledge Base

- **What it is:** A dictionary and database containing expert remedy information for all 23 plant classes.
- **What it does:** Normal image classifiers only output a raw label like 'Tomato_Early_blight'. This file translates that label into helpful advice: the name of the fungus, visual symptoms, preventive farming steps, organic eco-friendly sprays (like Neem oil), and chemical fungicides.
- **What to say to your professor:**

■ ■ *'This module converts our deep learning output into an actionable farmer advisory system, bridging raw computer vision with practical agricultural science.'*

■ 6. predict.py — Quick Command-Line Testing Tool

- **What it is:** A standalone script to test single leaf images in the terminal without opening a browser.
- **What it does:** You run `py predict.py --image path/to/leaf.jpg`, and it immediately outputs the prediction, confidence percentage, and remedies in the terminal.
- **What to say to your professor:**

■ ■ *'predict.py is our headless command-line inference script. It is useful for automated testing, batch processing, or integrating with external mobile APIs.'*

■ 7. run_app.bat & requirements.txt

- **run_app.bat:** A Windows batch script that allows anyone to double-click and launch the web app instantly.
- **requirements.txt:** Lists exact Python packages (torch, torchvision, streamlit, pillow, matplotlib) so anyone can install and run the project.

■ 8. The models/ Folder Files

- **plant_disease_model.pth:** The saved 'brain' of the model. It contains millions of learned numbers (weights and biases). That is why VS Code shows it as a binary file—it is meant to be loaded by PyTorch, not read as text.
- **class_names.json:** A list that matches class numbers (0 to 22) with real plant names.
- **training_metrics.png:** The saved graph showing training/validation loss dropping and accuracy rising across epochs.

3. Top 12 Viva Questions & Answers (Simple & Direct)

Study these questions with your group members. Professors frequently ask these exact concepts:

Q1: What is the main aim/objective of your project?

Answer: The main objective is to detect plant leaf diseases early and accurately using deep learning, and to provide farmers with immediate organic and chemical remedy recommendations through an easy-to-use web interface.

Q2: Why did you use a Convolutional Neural Network (CNN) instead of traditional Machine Learning (like Random Forest or SVM)?

Answer: Traditional machine learning requires manual feature extraction (measuring shapes, color histograms, texture manually), which fails when lighting or leaf orientation changes. CNNs automatically learn visual features (edges, spots, patterns) directly from raw image pixels layer by layer.

Q3: What is Transfer Learning, and why did you use MobileNetV3?

Answer: Transfer learning means taking a model already trained on millions of images (ImageNet) and fine-tuning it on our plant dataset. Instead of learning basic shapes from scratch, it already understands image fundamentals. We chose MobileNetV3 because it is lightweight, fast, and designed to run efficiently on standard computers without needing an expensive GPU.

Q4: What is the difference between Training Accuracy and Validation Accuracy?

Answer: Training accuracy measures how well the model predicts images it has already seen during practice. Validation accuracy measures how well the model performs on new, unseen test images. A high validation accuracy (like our 84.42%) proves that the model genuinely learned rather than just memorizing.

Q5: How did you prevent Overfitting in your model?

Answer: We used three main techniques: (1) Data Augmentation (random flips, rotations, and color jitter) so the model sees different variations of leaves; (2) A Dropout layer (0.3) in the classifier head to randomly turn off neurons and prevent over-reliance; and (3) A Validation split with checkpointing to save only the model with the highest validation score.

Q6: Why is the file 'plant_disease_model.pth' not readable in VS Code?

Answer: A .pth file is a serialized binary checkpoint containing millions of floating-point neural network weights and mathematical matrices. It is not plain text or Python code. It is loaded directly into computer memory by PyTorch using `torch.load()`.

Q7: What Loss Function and Optimizer did you use, and why?

Answer: We used Cross-Entropy Loss because it is the standard and most effective loss function for multi-class classification problems. For optimization, we used AdamW (Adam with decoupled weight decay), which dynamically adjusts learning rates for each parameter while preventing weights from growing too large.

Q8: What is the purpose of Data Augmentation in src/dataset.py?

Answer: In real life, farmers take photos under different sunlight, angles, and distances. Data augmentation artificially creates these conditions during training by randomly zooming, rotating, and flipping the leaves so the model performs reliably in real-world conditions.

Q9: What dataset did you use and how many classes are there?

Answer: We used the PlantVillage dataset containing over 35,000 images across 23 distinct classes. It covers major crops like Tomato, Potato, Apple, Corn, and Bell Pepper, including both healthy leaves and multiple fungal, bacterial, and viral diseases.

Q10: What are the inputs and outputs of your model?

Answer:

- Input: An RGB leaf photo resized and normalized to (3, 224, 224) pixels.
- Output: A probability distribution across 23 classes calculated using the Softmax activation function. The class with the highest probability is selected as the diagnosis.

Q11: Why is your project organized into multiple files instead of one big file?

Answer: Following software engineering best practices, we separated concerns into distinct modules: data loading, model architecture, training logic, advisory data, and UI. This makes the code readable, reusable, easier to debug, and simple to maintain.

Q12: What are the future enhancements for this project?

Answer:

1. Deploying as a native Android/iOS mobile app with offline inference.
2. Adding multi-language support (e.g. Hindi, Telugu, Tamil, Spanish) for local farmers.
3. Integrating severe weather alerts and pest risk prediction based on GPS location.

4. How to Present Your Demo in 2 Minutes

Follow these exact steps when presenting your screen to the evaluators:

Step 1: Double-click run_app.bat or run `py -m streamlit run app.py` in VS Code terminal.

Step 2: Open the browser at <http://localhost:8501>.

Step 3: Under 'Quick Demo', select **Tomato Early Blight** or **Apple Scab** from the sample dropdown.

Step 4: Show the confidence score (e.g. **99.46%**) and open the accordions to show the **Organic Remedies** (Neem oil, Bacillus subtilis) and **Chemical Sprays**.

Step 5: Switch to the '**Model Metrics & Curves**' tab to show the training accuracy graph, proving your model was trained and evaluated properly.